

■ LLM Comparison Lab

Run the Same Prompt Across GPT · Claude · Gemini

Compare Accuracy · Tone · Latency · Cost in Real-Time

AWS EC2

Jupyter / Code Server

Ubuntu t2.medium

Python 3.10+

AI / ML Hands-On Training Series · Fresher Track
Self-Contained Implementation Guide · Real-World Use Cases

TABLE OF CONTENTS

What's Inside This Lab

- [Section 1](#) Lab Overview & Real-World Use Case
- [Section 2](#) Architecture & Flow
- [Section 3](#) Environment Setup on AWS EC2
- [Section 4](#) Install Dependencies
- [Section 5](#) API Key Configuration
- [Section 6](#) Data Generation Script (50+ Test Prompts)
- [Section 7](#) Core LLM Caller Module
- [Section 8](#) Single-Prompt Comparison Notebook
- [Section 9](#) Batch Runner — All Prompts at Once
- [Section 10](#) Analysis & Visualization Notebook
- [Section 11](#) Results Interpretation Guide
- [Section 12](#) Bonus: Streamlit Dashboard
- [Section 13](#) Exercises & Submission Checklist

SECTION 01

Lab Overview & Real-World Use Case

What Are We Building?

In enterprise AI adoption, teams rarely use just one Large Language Model. Product managers, ML engineers, and AI architects must choose between GPT-4o (OpenAI), Claude 3.5 Sonnet (Anthropic), and Gemini 1.5 Pro (Google) based on accuracy, speed, cost, and tone. This lab automates that comparison — programmatically sending the same prompt to all three, capturing results, and producing a structured analysis report.

Real-World Scenario: TechCorp AI Procurement Team

TechCorp is evaluating which LLM to embed into their internal HR chatbot, customer support portal, and code review tool. They need hard data — not just vendor marketing. Your team has been tasked with building an automated LLM benchmarking harness that tests 50+ prompts across domains and produces a board-ready comparison report.

■ Learning Outcomes — By end of lab, you will be able to:

- Call OpenAI, Anthropic, and Google Gemini APIs with the same prompt
- Measure and record response latency per model
- Compare output quality across accuracy, tone, and length dimensions
- Generate test data covering HR, customer support, coding, and summarization use cases
- Build a batch runner and produce a CSV/HTML comparison report
- Visualize results using matplotlib and pandas

Domains Covered in This Lab

Domain	Example Use Case	Why It Matters
HR Assistant	Answer leave policy questions	Tone & accuracy critical
Customer Support	Handle refund complaints	Empathy + precision
Code Review	Review a Python function	Technical depth
Summarization	Summarize a financial report	Factual compression
Creative Writing	Write a product tagline	Tone & creativity
Reasoning / Math	Solve a word problem	Logic accuracy

SECTION 02

Architecture & Flow

Understanding the full pipeline before writing a single line of code is critical. Here is what this lab builds end-to-end:

Component	Description
1. Data Layer	prompts.json — 50+ categorised test prompts generated by script
2. Caller Module	llm_caller.py — unified function that calls all 3 LLM APIs
3. Batch Runner	batch_runner.ipynb — iterates prompts, collects responses + metadata
4. Results Store	results.csv — structured CSV: prompt, model, response, latency, tokens
5. Analysis Layer	analysis.ipynb — pandas + matplotlib charts
6. Dashboard	app.py (Streamlit) — interactive side-by-side viewer

Data Flow Diagram



SECTION 03

Environment Setup on AWS EC2

Your lab environment is a pre-provisioned AWS EC2 Ubuntu t2.medium instance. You access it via Jupyter Notebook (port 8888) or Code Server (VS Code in browser). Follow every step exactly — do not skip any.

Step 3.1 — Verify Your Instance Access

Open your browser and navigate to your assigned Jupyter URL:

```
# Format: http://<EC2-PUBLIC-IP>:8888
```

```
# Example:
```

```
http://54.123.45.67:8888
```

```
# Or Code Server (VS Code in browser):
```

```
http://54.123.45.67:8080
```

Step 3.2 — Open a Terminal Inside Jupyter

In Jupyter: click New → Terminal. In Code Server: click Terminal → New Terminal. All shell commands in this guide run inside this terminal.

Step 3.3 — Check Python Version

```
python3 --version
```

```
# Expected output: Python 3.10.x or 3.11.x
```

```
which python3
```

```
# Expected: /usr/bin/python3
```

Step 3.4 — Create Project Directory

```
cd ~
```

```
mkdir llm_comparison_lab
```

```
cd llm_comparison_lab
```

```
pwd
```

```
# Output: /home/ubuntu/llm_comparison_lab
```

Step 3.5 — Create a Python Virtual Environment

A virtual environment keeps your project's packages isolated from the system Python.

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
# Your terminal prompt should now show (venv):
```

```
# (venv) ubuntu@ip-172-xx:~/llm_comparison_lab$
```

```
# To deactivate later (don't do this now):
```

```
# deactivate
```

■ *Important: Every time you open a new terminal session, you must re-activate the venv with: source
~/llm_comparison_lab/venv/bin/activate*

SECTION 04

Install All Dependencies

Step 4.1 — Create requirements.txt

Run the command below to create the requirements file. Copy it exactly — each library is needed.

```
cat > requirements.txt << 'EOF'

openai==1.30.1

anthropic==0.26.1

google-generativeai==0.5.4

python-dotenv==1.0.1

pandas==2.2.2

matplotlib==3.9.0

seaborn==0.13.2

jupyter==1.0.0

ipywidgets==8.1.3

streamlit==1.35.0

plotly==5.22.0

tiktoken==0.7.0

tqdm==4.66.4

tabulate==0.9.0

EOF
```

Step 4.2 — Install All Packages

```
pip install --upgrade pip

pip install -r requirements.txt

# This takes 3-5 minutes. You will see progress bars.

# Final line should read: Successfully installed ...
```

Step 4.3 — Verify Key Installs

```
python3 -c "import openai; print('OpenAI OK:', openai.__version__)"

python3 -c "import anthropic; print('Anthropic OK:', anthropic.__version__)"

python3 -c "import google.generativeai as genai; print('Gemini OK:')"

python3 -c "import pandas as pd; print('Pandas OK:', pd.__version__)"
```

If any import fails, run: pip install and try again.

SECTION 05

API Key Configuration

All three LLM providers require API keys. Your trainer will provide the keys for this lab session. Never hardcode keys directly in scripts — use a .env file, which is the industry standard.

Step 5.1 — Where to Get API Keys

Provider	Portal URL	Free Tier?
OpenAI (GPT-4o)	platform.openai.com → API Keys	No (pay per token)
Anthropic (Claude)	console.anthropic.com → API Keys	No (pay per token)
Google (Gemini)	aistudio.google.com → Get API Key	Yes — limited

Step 5.2 — Create the .env File

```
# Inside ~/llm_comparison_lab/
```

```
nano .env
```

```
# Paste these lines (replace YOUR_KEY with actual keys):
```

```
OPENAI_API_KEY=sk-proj-YOUR_OPENAI_KEY_HERE
```

```
ANTHROPIC_API_KEY=sk-ant-YOUR_ANTHROPIC_KEY_HERE
```

```
GOOGLE_API_KEY=YOUR_GOOGLE_GEMINI_KEY_HERE
```

Save in nano: press `Ctrl+O` → `Enter` → `Ctrl+X`

Step 5.3 — Protect the .env File

```
chmod 600 .env
```

```
# This makes the file readable only by you (owner)
```

```
# Also create .gitignore to prevent accidental commits:
```

```
echo '.env' >> .gitignore
```

```
echo 'venv/' >> .gitignore
```

```
echo '__pycache__/' >> .gitignore
```

Step 5.4 — Test That Keys Load Correctly

```
python3 << 'EOF'
```

```
from dotenv import load_dotenv
```

```
import os
```

```
load_dotenv()
```

```
keys = ['OPENAI_API_KEY', 'ANTHROPIC_API_KEY', 'GOOGLE_API_KEY']
```

```
for k in keys:
    val = os.getenv(k, '')
    status = '✓ Set' if val else '✗ MISSING'
    print(f'{k}: {status} ({len(val)} chars)')

EOF
```

Expected output: all three keys show '✓ Set' with a non-zero character count. If any key shows MISSING, re-open .env and check for typos.

Data Generation Script — 50+ Test Prompts

Step 6.1 — Create generate_prompts.py

```
nano generate_prompts.py
```

Then paste the entire script below and save (Ctrl+O → Enter → Ctrl+X)

```
"""  
generate_prompts.py  
  
Creates a diverse set of 50+ test prompts across enterprise domains.  
  
Output: prompts.json  
  
""">  
  
import json  
  
PROMPTS = [  
  
# ■■■ DOMAIN 1: HR ASSISTANT (10 prompts) ■■■■■■■■■■■■■■■■■■■■  
  
{ "id": "HR001", "domain": "HR Assistant", "difficulty": "easy",  
  "prompt": "How many casual leaves am I entitled to per year?" },  
  
{ "id": "HR002", "domain": "HR Assistant", "difficulty": "easy",  
  "prompt": "Can I carry forward unused earned leaves to the next year?" },  
  
{ "id": "HR003", "domain": "HR Assistant", "difficulty": "medium",  
  "prompt": "What is the process to apply for maternity leave and what documents are needed?" },  
  
{ "id": "HR004", "domain": "HR Assistant", "difficulty": "medium",  
  "prompt": "My manager rejected my leave request for Diwali. What are my options?" },  
  
{ "id": "HR005", "domain": "HR Assistant", "difficulty": "hard",  
  "prompt": "Explain the company policy on moonlighting and dual employment." },  
  
{ "id": "HR006", "domain": "HR Assistant", "difficulty": "medium",  
  "prompt": "How do I apply for a salary advance for medical emergency?" },  
  
{ "id": "HR007", "domain": "HR Assistant", "difficulty": "easy",  
  "prompt": "What are the office working hours and are there flexible timing options?" },  
  
{ "id": "HR008", "domain": "HR Assistant", "difficulty": "hard",
```

```

{"prompt": "Explain the Performance Improvement Plan (PIP) process and my rights during it."},
{"id": "HR009", "domain": "HR Assistant", "difficulty": "medium",
{"prompt": "How do I file a complaint about workplace harassment anonymously?"},
{"id": "HR010", "domain": "HR Assistant", "difficulty": "easy",
{"prompt": "What is the notice period for resignation and can it be negotiated?"},

```

[illegible]

```
{
  "id": "CS001",
  "domain": "Customer Support",
  "difficulty": "easy",
  "prompt": "I ordered a laptop 5 days ago and it hasn't arrived. Help me track it."
},
{
  "id": "CS002",
  "domain": "Customer Support",
  "difficulty": "medium",
  "prompt": "The product I received is damaged. I want a refund but the seller is not responding."
},
```

```
{
  "id": "CS003",
  "domain": "Customer Support",
  "difficulty": "hard",
  "prompt": "I was charged twice for the same order last month. My bank also shows double debit. I want an immediate resolution.",
}
```

```
{
  "id": "CS004",
  "domain": "Customer Support",
  "difficulty": "easy",
  "prompt": "How do I cancel my premium subscription and get a prorated refund?"
},
{
  "id": "CS005",
  "domain": "Customer Support",
  "difficulty": "medium",
  "prompt": "How do I cancel my premium subscription and get a prorated refund?"
}
```

```
"prompt": "I received the wrong color of the product. The return window shows closed but I received it yesterday."},
```

```
{"id": "CS006", "domain": "Customer Support", "difficulty": "hard",  
"prompt": "Your delivery partner broke my TV during installation and now nobody is taking  
responsibility. Escalate this."},
```

```
{
  "id": "CS007",
  "domain": "Customer Support",
  "difficulty": "easy",
  "prompt": "What is your return policy for electronics?"
},
{
  "id": "CS008",
  "domain": "Customer Support",
  "difficulty": "medium",
  "prompt": "How do I track my order?"
}
```

```
"prompt": "I used a coupon code during checkout but the discount was not applied. I already placed the order."},
```

```
{
  "id": "CS009",
  "domain": "Customer Support",
  "difficulty": "medium",
  "prompt": "My account has been suspended without notice. I have an active order. Help urgently."
}
```

```
{
  "id": "CS010",
  "domain": "Customer Support",
  "difficulty": "hard",
  "prompt": "I am a visually impaired customer and your app is completely inaccessible. This is a legal issue."
}
```

[illegible]

```
{
  "id": "CR001",
  "domain": "Code Review",
  "difficulty": "easy",
  "prompt": "Review this Python function for bugs: def add(a, b): return a - b",
  "analysis": "The function 'add' is named incorrectly for its operation. It subtracts 'b' from 'a' instead of adding them. The prompt asks to review for bugs, and this is a clear logic error."
},
{
  "id": "CR002",
  "domain": "Code Review",
  "difficulty": "medium",
  "prompt": "Review this JavaScript function for bugs: function sum(a, b) { return a + b; }",
  "analysis": "The function 'sum' is correctly named and implements addition. However, the prompt asks to review for bugs, and there are no obvious bugs in the provided code snippet."
}
```

```
"prompt": "What is wrong with this SQL query: SELECT * FROM users WHERE id = ' + user_id + '",
```

```
{"id": "CR003", "domain": "Code Review", "difficulty": "hard",
```

```
"prompt": "Review this code for thread safety issues: class Counter: count=0\n def increment(self): self.count += 1"},
```

```
{"id": "CR004", "domain": "Code Review", "difficulty": "medium",
```

```
"prompt": "Is this a memory leak? def process(): data=[]; while True: data.append(get_data())"},
```

```
{"id": "CR005", "domain": "Code Review", "difficulty": "easy",
```

```
"prompt": "What design pattern should I use to avoid many if-elif chains in my Python code?"},
```

```
{"id": "CR006", "domain": "Code Review", "difficulty": "hard",
```

```
"prompt": "Explain the time complexity of this code: for i in range(n): for j in range(n): for k in range(n): pass"},
```

```
{"id": "CR007", "domain": "Code Review", "difficulty": "medium",
```

```
"prompt": "How do I refactor this: def get_user_data(id): import requests; return requests.get(url+id).json()"},
```

```
{"id": "CR008", "domain": "Code Review", "difficulty": "easy",
```

```
"prompt": "When should I use a list vs a tuple vs a set in Python?"},
```

```
{"id": "CR009", "domain": "Code Review", "difficulty": "hard",
```

```
"prompt": "Explain and fix the GIL problem in this multi-threaded Python CPU-bound code."},
```

```
{"id": "CR010", "domain": "Code Review", "difficulty": "medium",
```

```
"prompt": "Is it better to use async/await or threads for I/O-bound Python tasks? Explain with example."},
```

```
# ■■■ DOMAIN 4: SUMMARIZATION (10 prompts) ■■■■■■■■■■■■■■■■■■■■
```

```
{"id": "SU001", "domain": "Summarization", "difficulty": "easy",
```

```
"prompt": "Summarize in 3 bullet points: The RBI raised repo rate by 25 bps to 6.75%. This is the 8th consecutive hike. EMIs on home loans will increase by Rs 400 per lakh."},
```

```
{"id": "SU002", "domain": "Summarization", "difficulty": "medium",
```

```
"prompt": "Summarize this for a non-technical CEO: Our microservices architecture experienced cascading failures due to missing circuit breakers resulting in 4 hours of downtime."},
```

```
{"id": "SU003", "domain": "Summarization", "difficulty": "hard",
```

```
"prompt": "Create an executive summary of Q3 results: Revenue Rs 2400Cr (+18% YoY), EBITDA Rs 480Cr margin 20%, PAT Rs 220Cr, Cash Rs 800Cr, Debt reduced 30%, Guidance revised upward."},
```

```
{"id": "SU004", "domain": "Summarization", "difficulty": "easy",
```

```
"prompt": "Summarize the key risks: inflation rising, dollar strengthening, geopolitical tensions, supply chain issues, interest rate hikes expected."},
```

```
{"id": "SU005", "domain": "Summarization", "difficulty": "medium",
```

```
"prompt": "Turn this into a tweet thread (5 tweets): India's EV market grew 120% in 2023. Two-wheelers dominate with 55% share. Top brand Ola Electric has 28% market share. Government FAME-2 subsidy extended to 2025."},
```

```
{"id": "SU006", "domain": "Summarization", "difficulty": "easy",
```



```

def main():

output_path = "prompts.json"

with open(output_path, "w") as f:

json.dump(PROMPTS, f, indent=2)

print(f"Generated {len(PROMPTS)} prompts → {output_path}")

domains = {}

for p in PROMPTS:

d = p["domain"]

domains[d] = domains.get(d, 0) + 1

print("\nBreakdown:")

for d, c in domains.items():

print(f" {d}: {c} prompts")


if __name__ == "__main__":

main()

```

Step 6.2 — Run the Data Generator

```

python3 generate_prompts.py

# Expected output:

# Generated 60 prompts → prompts.json

# Breakdown:

# HR Assistant: 10 prompts

# Customer Support: 10 prompts

# Code Review: 10 prompts

# Summarization: 10 prompts

# Creative Writing: 10 prompts

# Reasoning: 10 prompts


# Verify the file was created:

ls -la prompts.json

python3 -c "import json; d=json.load(open('prompts.json')); print(len(d), 'prompts loaded')"

```

Core LLM Caller Module

Step 7.1 — Create llm_caller.py

Page 16 | AWS EC2 · Jupyter · Code Server

```
messages.append({"role": "user", "content": prompt})
```

```
start = time.time()
```

```
try:
```

```
resp = openai_client.chat.completions.create(
```

```
model=MODELS["gpt"],
```

```
messages=messages,
```

```
max_tokens=500,
```

```
temperature=0.7,
```

```
)
```

```
latency = round(time.time() - start, 3)
```

```
return {
```

```
"model": "GPT-4o",
```

```
"response": resp.choices[0].message.content.strip(),
```

```
"latency_s": latency,
```

```
"input_tokens": resp.usage.prompt_tokens,
```

```
"output_tokens": resp.usage.completion_tokens,
```

```
"error": None,
```

```
}
```

```
except Exception as e:
```

```
return {"model": "GPT-4o", "response": "", "latency_s": -1,
```

```
"input_tokens": 0, "output_tokens": 0, "error": str(e)}
```

```
def call_claude(prompt: str, system: str = "") -> dict:
```

```
    """Call Anthropic Claude 3.5 Sonnet and return standardised result."""
```

```
start = time.time()
```

```
try:
```

```
resp = anthropic_client.messages.create(
```

```
model=MODELS["claude"],
```

```
max_tokens=500,
```

```
system=system or "You are a helpful AI assistant.",
```

```
messages=[{"role": "user", "content": prompt}],
```

```
)
```

```
latency = round(time.time() - start, 3)
```

```
return {
```

```
"model": "Claude-3.5-Sonnet",
```

```

"response": resp.content[0].text.strip(),
"latency_s": latency,
"input_tokens": resp.usage.input_tokens,
"output_tokens": resp.usage.output_tokens,
"error": None,
}

except Exception as e:

return {"model": "Claude-3.5-Sonnet", "response": "", "latency_s": -1,
"input_tokens": 0, "output_tokens": 0, "error": str(e)}

def call_gemini(prompt: str, system: str = "") -> dict:
    """Call Google Gemini 1.5 Pro and return standardised result."""
    full_prompt = f"{system}\n\n{prompt}" if system else prompt
    start = time.time()
    try:
        resp = gemini_model.generate_content(
            full_prompt,
            generation_config=genai.types.GenerationConfig(
                max_output_tokens=500,
                temperature=0.7,
            )
        )
        latency = round(time.time() - start, 3)
        text = resp.text.strip() if resp.text else ""
        # Gemini does not always return token counts
        in_tok = getattr(resp.usage_metadata, "prompt_token_count", 0)
        out_tok = getattr(resp.usage_metadata, "candidates_token_count", 0)
        return {
            "model": "Gemini-1.5-Pro",
            "response": text,
            "latency_s": latency,
            "input_tokens": in_tok,
            "output_tokens": out_tok,
            "error": None,
        }

```

```

except Exception as e:

return {"model": "Gemini-1.5-Pro", "response": "", "latency_s": -1,
"input_tokens": 0, "output_tokens": 0, "error": str(e)}

def compare_all(prompt: str, system: str = "") -> list[dict]:
"""
Run the same prompt across all 3 LLMs.
Returns list of 3 result dicts with identical schema.
"""
results = []
for caller in [call_gpt, call_claude, call_gemini]:
result = caller(prompt, system)
result["prompt"] = prompt
results.append(result)
return results

if __name__ == "__main__":
# Quick smoke test
test_prompt = "In one sentence, what is machine learning?"
print("Testing all 3 LLMs with:", test_prompt)
print("-" * 60)
for r in compare_all(test_prompt):
print(f"[{r['model']}] {r['latency_s']}s")
print(r["response"])
print()

```

Step 7.2 — Smoke Test the Caller Module

```

python3 llm_caller.py

# Expected output (actual responses will vary):
# Testing all 3 LLMs with: In one sentence, what is machine learning?
# [GPT-4o] 1.23s
# Machine learning is a subset of AI where systems learn from data...
# [Claude-3.5-Sonnet] 1.87s
# Machine learning is the field of AI that enables computers to learn...

```

```
#
```

```
# [Gemini-1.5-Pro] 2.10s
```

```
# Machine learning is a branch of AI that allows systems to improve...
```

SECTION 08

Single-Prompt Comparison Notebook

This Jupyter notebook lets you type any custom prompt, run it against all three models, and see a formatted side-by-side comparison. It is the quickest way to do ad-hoc testing during the lab.

Step 8.1 — Create the Notebook File

```
# Open Jupyter in your browser

# Click: New → Python 3 (ipykernel)

# Rename the notebook to: 01_single_prompt_compare.ipynb

# Then enter each cell below in order
```

Notebook Cell 1 — Imports and Setup

```
# Cell 1 – Run this first

import sys

sys.path.insert(0, '.')

from llm_caller import compare_all

import pandas as pd

from IPython.display import display, HTML

import warnings

warnings.filterwarnings('ignore')

print('Setup complete. All libraries loaded.')
```

Notebook Cell 2 — Define Display Function

```
# Cell 2 – Pretty display helper

def show_comparison(results):

    html = '<table border=1 style="width:100%;border-collapse:collapse">'

    html += '<tr style="background:#1a1a2e;color:white">'

    for r in results:

        html += f'<th style="padding:10px">{r["model"]}</th>'

    html += '</tr>'

    # Latency row

    html += '<tr style="background:#0f3460">'

    for r in results:

        html += f'<td style="padding:8px;color:#e94560">■ {r["latency_s"]}s</td>'

    html += '</tr>'
```

```
# Token count row

html += '<tr style="background:#16213e">'

for r in results:

html += f'<td style="padding:8px;color:#a8a8b3">■ {r["output_tokens"]} tokens out</td>'

html += '</tr>'

# Response row

html += '<tr>'

for r in results:

resp = r['response'].replace('\n', '<br>') if r['response'] else f'<span
style="color:red">Error: {r["error"]}</span>'

html += f'<td style="padding:10px;vertical-align:top">{resp}</td>'

html += '</tr></table>'

display(HTML(html))

print('Display function ready.')
```

Notebook Cell 3 — Run Your Prompt

```
# Cell 3 – Change this prompt and re-run to test different queries

MY_PROMPT = """

A customer says: 'I ordered a phone 7 days ago, it hasn't arrived,
and your support team hasn't responded to my 3 emails. I am furious.'

Write a professional, empathetic customer support reply.

"""

print(f'Sending to 3 LLMs: {MY_PROMPT[:60]}...')

print('Please wait...')

results = compare_all(MY_PROMPT)

print('Done! Rendering comparison...')

show_comparison(results)
```

Notebook Cell 4 — Export to DataFrame

```
# Cell 4 – Convert to pandas for easy analysis

df = pd.DataFrame(results)

df_display = df[['model', 'latency_s', 'input_tokens', 'output_tokens']]

print('Metrics Summary:')

display(df_display)

# Fastest model

fastest = df.loc[df['latency_s'].idxmin(), 'model']
```

```
print(f'\nFastest model: {fastest}')
```

```
# Longest response
```

```
longest = df.loc[df['output_tokens'].idxmax(), 'model']
```

```
print(f'Most verbose model: {longest}')
```

SECTION 09

Batch Runner — All Prompts at Once

The batch runner iterates over all 60 prompts in `prompts.json`, calls all three LLMs for each prompt, and saves every response to `results.csv`. This is the data collection phase — it may take 15–25 minutes to complete.

■ *Cost Note: Running 60 prompts × 3 models = 180 API calls. Estimated cost: ~\$0.50–\$1.50 total depending on token lengths. Your trainer has pre-loaded the lab accounts with sufficient credits.*

Step 9.1 — Create `02_batch_runner.ipynb`

```
# Create new notebook: 02_batch_runner.ipynb
```

```
# Add the following cells in order:
```

Batch Runner — Cell 1: Setup

```
import json, csv, os, time

import sys; sys.path.insert(0, '.')

from llm_caller import compare_all

from tqdm.notebook import tqdm

import pandas as pd

# Load prompts

with open('prompts.json') as f:
    prompts = json.load(f)

print(f'Loaded {len(prompts)} prompts')
print('Domains:', set(p['domain'] for p in prompts))
```

Batch Runner — Cell 2: Run and Save

```
RESULTS_FILE = 'results.csv'

FIELDNAMES = ['prompt_id', 'domain', 'difficulty', 'prompt',
              'model', 'response', 'latency_s',
              'input_tokens', 'output_tokens', 'error']

# Open CSV and write headers

with open(RESULTS_FILE, 'w', newline='', encoding='utf-8') as csvfile:
    writer = csv.DictWriter(csvfile, fieldnames=FIELDNAMES)
    writer.writeheader()

    for item in tqdm(prompts, desc='Processing prompts'):
        results = compare_all(item['prompt'])
```

```

for r in results:
    row = {
        'prompt_id': item['id'],
        'domain': item['domain'],
        'difficulty': item['difficulty'],
        'prompt': item['prompt'],
        'model': r['model'],
        'response': r['response'],
        'latency_s': r['latency_s'],
        'input_tokens': r['input_tokens'],
        'output_tokens': r['output_tokens'],
        'error': r['error'],
    }
    writer.writerow(row)
    # Small delay to avoid rate limiting
    time.sleep(1.5)

print(f'All done! Results saved to {RESULTS_FILE}')

```

Batch Runner — Cell 3: Quick Sanity Check

```

df = pd.read_csv('results.csv')
print(f'Total rows: {len(df)}')
print(f'Models: {df["model"].unique()}')
print(f'Domains: {df["domain"].unique()}')
print(f'Errors: {df["error"].notna().sum()}')
print('\nFirst 3 rows:')
display(df[['prompt_id', 'model', 'latency_s', 'output_tokens']].head(3))

```

SECTION 10

Analysis & Visualization Notebook

Now we analyse results.csv using pandas and matplotlib to uncover patterns: which model is fastest, most verbose, and highest quality per domain.

Step 10.1 — Create 03_analysis.ipynb

```
# Create new notebook: 03_analysis.ipynb
```

Analysis Cell 1 — Load and Inspect

```
import pandas as pd

import matplotlib.pyplot as plt

import matplotlib

import seaborn as sns

matplotlib.rcParams['figure.facecolor'] = '#0D1117'
matplotlib.rcParams['axes.facecolor'] = '#161B22'
matplotlib.rcParams['text.color'] = 'white'
matplotlib.rcParams['axes.labelcolor'] = 'white'
matplotlib.rcParams['xtick.color'] = 'white'
matplotlib.rcParams['ytick.color'] = 'white'

df = pd.read_csv('results.csv')

# Filter only successful rows
df = df[df['error'].isna()].copy()

df['response_len'] = df['response'].apply(len)

print(df.shape)

display(df.describe())
```

Analysis Cell 2 — Latency Comparison Chart

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

fig.suptitle('LLM Latency Comparison', color='white', fontsize=14, fontweight='bold')

# Bar chart: Average latency per model
avg_lat = df.groupby('model')['latency_s'].mean().sort_values()

colors = ['#238636', '#1F6FEB', '#E3B341']

avg_lat.plot(kind='bar', ax=axes[0], color=colors, rot=15)

axes[0].set_title('Avg Latency (seconds)', color='white')

axes[0].set_ylabel('Seconds')
```

```

for bar, val in zip(axes[0].patches, avg_lat):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.05,
f'{val:.2f}s', ha='center', color='white', fontsize=9)

# Box plot: Latency distribution
models = df['model'].unique()
data_for_box = [df[df['model']==m]['latency_s'].values for m in models]
bp = axes[1].boxplot(data_for_box, labels=models, patch_artist=True)
for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)
axes[1].set_title('Latency Distribution', color='white')
axes[1].set_ylabel('Seconds')

plt.tight_layout()
plt.savefig('chart_latency.png', dpi=150, bbox_inches='tight')
plt.show()
print('Chart saved: chart_latency.png')

```

Analysis Cell 3 — Response Length by Domain

```

fig, ax = plt.subplots(figsize=(14, 6))
pivot = df.groupby(['domain', 'model'])['output_tokens'].mean().unstack()
pivot.plot(kind='bar', ax=ax,
color=['#238636', '#1F6FEB', '#E3B341'],
width=0.7, rot=20)
ax.set_title('Avg Output Tokens by Domain & Model', color='white', fontsize=13)
ax.set_xlabel('Domain')
ax.set_ylabel('Avg Output Tokens')
ax.legend(title='Model')
plt.tight_layout()
plt.savefig('chart_tokens_by_domain.png', dpi=150, bbox_inches='tight')
plt.show()

```

Analysis Cell 4 — Summary Statistics Table

```

summary = df.groupby('model').agg(
    avg_latency = ('latency_s', 'mean'),
    min_latency = ('latency_s', 'min'),
    max_latency = ('latency_s', 'max'),
    avg_out_tokens = ('output_tokens', 'mean'),

```

```
total_calls = ('model', 'count'),  
) .round(2)  
  
print('=== SUMMARY STATISTICS ===')  
display(summary)  
summary.to_csv('summary_stats.csv')  
print('Saved: summary_stats.csv')
```

SECTION 11

Results Interpretation Guide

Raw numbers need context. Here is how to interpret your benchmark results and translate them into actionable recommendations.

Latency Benchmarks (Expected Ranges)

Metric	GPT-4o	Claude 3.5 Sonnet	Gemini 1.5 Pro
Avg Latency (short)	0.8–1.5s	1.2–2.0s	1.5–2.5s
Avg Latency (long)	2.0–4.0s	2.5–5.0s	2.5–5.0s
Token/sec (output)	~80–120	~60–100	~70–110
Cost per 1M tokens	\$5 in / \$15 out	\$3 in / \$15 out	\$3.5 in / \$10.5 out

Quality Interpretation Framework

■ How to Score Response Quality
• Accuracy: Does the answer correctly address the question? (1-5 scale) • Completeness: Does it cover all relevant aspects without being asked twice? • Tone: Is the tone appropriate for the domain (formal for HR, empathetic for CS)? • Conciseness: Does it avoid waffle and get to the point? • Structure: Is it well-formatted (bullets, headers) when needed?

Typical Observed Patterns

Observation	Likely Cause	Recommendation
GPT fastest for short prompts	Optimised batching infra	Use GPT for real-time chat
Claude longest responses	Model trained for verbosity	Use Claude for detailed docs
Gemini best for structured output	Google training data mix	Use Gemini for JSON/tables
All models similar accuracy on easy prompts	Saturation at low difficulty	Use hard prompts for benchmarking
High latency variance	Network / API queue effects	Run 3+ trials and average

SECTION 12

Bonus: Streamlit Interactive Dashboard

This Streamlit app lets you type any prompt, see all three LLM responses side-by-side, and explore the batch results interactively — all in your browser.

Step 12.1 — Create app.py

```
nano app.py

import streamlit as st

import sys; sys.path.insert(0, ".")

from llm_caller import compare_all

import pandas as pd

import plotly.express as px


st.set_page_config(page_title="LLM Battle Arena",
page_icon="🎮", layout="wide")


st.title("🎮 LLM Battle Arena")
st.caption("Compare GPT-4o vs Claude 3.5 vs Gemini 1.5 Pro in real-time")


tab1, tab2 = st.tabs(["🎮 Live Compare", "📊 Batch Results"])


with tab1:

    prompt = st.text_area("Enter your prompt:", height=120,
placeholder="Type any question or task here...")

    if st.button("🎮 Run on all 3 LLMs", type="primary"):

        if not prompt.strip():

            st.warning("Please enter a prompt first.")

        else:

            with st.spinner("Calling GPT, Claude and Gemini..."):

                results = compare_all(prompt)

                cols = st.columns(3)

                for col, r in zip(cols, results):

                    with col:

                        st.subheader(r["model"])

                        st.metric("Latency", f'{r["latency_s"]}s')

                        st.metric("Output tokens", r["output_tokens"])
```

```

st.divider()

if r["error"]:
    st.error(f'Error: {r["error"]}')
else:
    st.write(r["response"])

with tab2:
    try:
        df = pd.read_csv("results.csv")
        df = df[df["error"].isna()]
        st.subheader("Latency by Model")
        fig = px.box(df, x="model", y="latency_s", color="model",
                    title="Latency Distribution")
        st.plotly_chart(fig, use_container_width=True)
        st.subheader("Output Tokens by Domain")
        fig2 = px.bar(df.groupby(["domain", "model"])["output_tokens"]
                    .mean().reset_index(),
                    x="domain", y="output_tokens", color="model",
                    barmode="group", title="Avg Output Tokens by Domain")
        st.plotly_chart(fig2, use_container_width=True)
        st.subheader("Raw Results")
        st.dataframe(df, use_container_width=True)
    except FileNotFoundError:
        st.info("Run the batch notebook first to generate results.csv")

```

Step 12.2 — Launch the Dashboard

```

# In terminal (with venv active):

streamlit run app.py --server.port 8501 --server.address 0.0.0.0

# Access in your browser:

# http://<EC2-PUBLIC-IP>:8501

# Note: Port 8501 must be open in your EC2 Security Group.

# Ask your trainer if you cannot access it.

```

SECTION 13

Exercises & Submission Checklist

Exercise 1 — Custom Prompt Battle

■ Task

- Write 5 of your own prompts (at least 1 per domain)
- Run them through the single-prompt notebook
- Screenshot the comparison table for each
- Write 2–3 sentences per prompt: which model won and why?

Exercise 2 — System Prompt Experiment

■ Task

- Modify `llm_caller.py` to pass a custom system prompt: 'You are a strict corporate legal advisor. Be concise and use formal language.'
- Re-run the HR domain prompts (HR001–HR010)
- Compare: does the system prompt significantly change tone? Which model follows it best?
- Create a table: `prompt_id` | `without_system` | `with_system` | `difference`

Exercise 3 — Latency Under Load

■ Task

- Run the same prompt 10 times to each model using a loop
- Collect all latency values and compute mean, std, min, max
- Plot a line chart showing latency across 10 trials for each model
- Which model has the most consistent latency (lowest std)?

Submission Checklist

■	<code>prompts.json</code> generated with 60 prompts
■	<code>llm_caller.py</code> smoke test passes for all 3 models
■	<code>01_single_prompt_compare.ipynb</code> runs without errors
■	<code>02_batch_runner.ipynb</code> completes and <code>results.csv</code> exists

■	03_analysis.ipynb produces at least 2 charts
■	Streamlit dashboard launches and shows live comparisons
■	Exercise 1: 5 custom prompts compared with written analysis
■	Exercise 2: System prompt experiment table completed
■	Exercise 3: Latency under load chart saved
■	summary_stats.csv exported and shared with trainer

Quick Reference — Key Commands

Command	Purpose
<code>source venv/bin/activate</code>	Activate Python virtual environment
<code>python3 generate_prompts.py</code>	Generate 60 test prompts → prompts.json
<code>python3 llm_caller.py</code>	Smoke test all 3 LLM connections
<code>jupyter notebook</code>	Launch Jupyter (port 8888)
<code>streamlit run app.py --server.port 8501 --server.address 0.0.0.0</code>	Launch dashboard
<code>cat results.csv wc -l</code>	Count rows in results file
<code>python3 -c "import pandas as pd; print(pd.read_csv('results.csv').shape)"</code>	Check results shape

Congratulations on completing the LLM Comparison Lab! You now have a production-grade benchmarking harness that any enterprise AI team would use to make data-driven LLM procurement decisions. The skills — API integration, structured data collection, batch processing, and visualization — are directly applicable to real-world AI engineering roles.

Next Steps: Explore LangChain for model-agnostic prompt routing, try LiteLLM for a unified API across 100+ models, or build a RAG system and benchmark how each LLM performs on retrieval-augmented tasks.